RoomRent Production Security Checklist

Project-specific pre-production measures for the Flask website and admin service

Generated: 2026-05-19

This document summarizes the production security measures recommended for the RoomRent Flask website and admin service. The highest-risk items should be fixed before exposing the application to real users or public traffic.

Immediate Launch Blockers

- Move secrets out of Flask client-side sessions. The app stores OTP state, admin signup/reset data, admin secret keys, and MFA setup secrets in session in app.py and adminsApp.py. Flask sessions are signed cookies, not encrypted; users can read their own cookie contents. Store sensitive temporary auth state in Redis, MySQL, or another server-side session store.

- Disable demo OTPs completely. OTP_DEMO_MODE is enabled and OTPs are shown in the UI. Admin OTPs are also displayed for signup and reset flows. Use a real SMS or email provider and never display OTPs in the browser or logs.

- Add CSRF protection to every POST form. Login, signup, inquiry, admin login, MFA, reset password, and MFA disable forms need CSRF tokens. Use Flask-WTF CSRFProtect or equivalent signed CSRF middleware.

- Harden cookies and HTTPS settings for both apps: SESSION_COOKIE_SECURE=True, SESSION_COOKIE_HTTPONLY=True, SESSION_COOKIE_SAMESITE='Lax' or 'Strict' for admin, and PREFERRED_URL_SCHEME='https'. HttpOnly and SameSite are present, but Secure is missing and the scheme is currently http.

- Use strong production secrets and rotate local/dev values. Set fresh long random values for SECRET_KEY, ADMIN_APP_SECRET, Google OAuth secrets, ADMIN_SIGNUP_CODE, DB password, admin password, and admin secret key. Rotate anything that was shared, committed, pasted, or exposed.

Authentication And Admin Controls

- Protect admin access. Put the admin service behind VPN, IP allowlist, private network, or an authenticated reverse proxy. Require MFA for every admin account.

- Set ADMIN_SIGNUP_CODE in production. Better: disable public admin signup after the first admin account is created and use a controlled admin provisioning process.

- Add rate limiting and lockouts. Protect /login, /verify-otp, admin login, admin reset, MFA, inquiry submission, and API endpoints. Use per-IP and per-account limits backed by Redis or another shared store.

- Use generic auth errors for users while logging detailed causes server-side. Avoid exposing whether an account, phone, admin ID, or secret key exists.

Database And Storage

- Fail closed if MySQL is unavailable. In production, database unavailability should return an error or stop startup. It should not continue with demo data or JSON fallback storage.

- Do not use MySQL root in production. Create a least-privilege application database user with only the permissions the app needs.

- Restrict database network access to the application host or private subnet. Do not expose MySQL directly to the public internet.

- Enable automated encrypted backups, test restore, and define retention. Backups are part of the security plan because they limit ransomware/data-loss impact.

Deployment Hardening

- Do not run python app.py or python adminsApp.py directly on the internet. Use a production WSGI server such as Waitress, Gunicorn, or uWSGI behind Nginx, Apache, Caddy, or the hosting platform's proxy.

- Keep FLASK_DEBUG=0 in production. Flask warns that the debugger can execute arbitrary Python code from the browser and must not be exposed.

- Terminate HTTPS at the proxy or platform, redirect HTTP to HTTPS, and enable HSTS once HTTPS is confirmed working.

- Configure request size limits, timeouts, worker limits, process restarts, health checks, and service monitoring.

- Set trusted hosts/proxy handling correctly so url_for(..., _external=True), OAuth redirects, and Host header handling cannot be abused.

Browser Security Headers

- Add Content-Security-Policy. Start with a restrictive policy and explicitly allow only the scripts, styles, images, fonts, and API origins the app actually uses.

- Add Strict-Transport-Security after HTTPS is stable: for example max-age=31536000; includeSubDomains, if all subdomains support HTTPS.

- Add X-Content-Type-Options: nosniff.

- Add frame protection with CSP frame-ancestors 'self' or X-Frame-Options: SAMEORIGIN/DENY to reduce clickjacking risk.

- Add Referrer-Policy such as strict-origin-when-cross-origin or no-referrer, depending on product needs.

- Flask-Talisman can manage many of these headers, but review the final headers in a browser and scanner before launch.

Input, Output, And API Controls

- Keep SQL parameterized. The current database code appears to use parameterized queries for user inputs; preserve that pattern.

- Validate and bound inputs. Enforce length and format limits for names, emails, phone numbers, inquiry messages, filters, OTP fields, and admin IDs.

- Set Flask request limits such as MAX_CONTENT_LENGTH and review form limits. This helps reduce denial-of-service risk from oversized requests.

- Keep Jinja autoescaping on and avoid Markup, render_template_string, or |safe for user-provided data unless it is carefully sanitized.

- Return only the data each API caller needs. Avoid exposing owner phone numbers, user activity, IP addresses, or analytics to unauthorized users.

Logging, Privacy, And Monitoring

- Avoid logging secrets, OTPs, session cookies, OAuth tokens, passwords, reset tokens, or admin secret keys.

- You collect user IP address, user agent, referrer, and activity footprints. Add a retention policy and privacy notice before production.

- Add security logging for login failures, OTP failures, admin reset attempts, MFA failures, rate-limit events, database errors, and privilege changes.

- Monitor errors and suspicious activity. Alert on repeated login failures, unusual admin activity, high request volume, and database fallback/errors.

- Prepare an incident response process: who responds, how secrets are rotated, how users are notified, and how backups are restored.

Dependency And Supply Chain

- Pin dependencies and regularly scan them with pip-audit, Safety, GitHub Dependabot, or an equivalent tool.

- Build and deploy from a clean source repository. Do not deploy local .env files, instance files, logs, __pycache__, or virtualenv folders.

- Keep .env out of version control. The current .gitignore includes .env and instance/, which is good; verify the real repository has not already tracked them.

- Use separate dev, staging, and production secrets. Never reuse local credentials in production.

Current Good Signs

- SQL access appears to use parameterized queries rather than directly interpolating request inputs.

- Admin passwords are hashed using Werkzeug password hashing, not stored as plaintext.

- .env and instance/ are listed in .gitignore.

- Debug mode defaults to off unless FLASK_DEBUG=1 is set.

Reference Links

- Flask security docs: https://flask.palletsprojects.com/en/stable/web-security/

- Flask sessions docs: https://flask.palletsprojects.com/en/stable/quickstart/#sessions

- OWASP CSRF Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

- OWASP HTTP Security Headers Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html

- OWASP Session Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

- OWASP Top Ten: https://owasp.org/www-project-top-ten/